Ruhan Garg

Roll No: 1024170291

EWA (UCS553) – Week 5

List of Experiments: Exception & Debugging – Java Solutions

Q1. Program demonstrating the hierarchy of common exceptions (ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException)

```
import java.util.Scanner;

public class ExceptionHierarchyDemo {

    static void demoArithmetic() {
        try {
            int a = 10, b = 0;
            int result = a / b;              // ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught: " + e.getMessage());
        }
    }

    static void demoNullPointer() {
        try {
            String s = null;
            System.out.println(s.length());  // NullPointerException
        } catch (NullPointerException e) {
            System.out.println("NullPointerException caught: " + e.getMessage());
        }
    }

    static void demoArrayIndex() {
        try {
            int[] arr = {1, 2, 3};
            System.out.println(arr[5]);       // ArrayIndexOutOfBoundsException
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException caught: " + e.getMessage());
        }
    }

    static void demoNumberFormat() {
        try {
            String num = "abc123";
            int value = Integer.parseInt(num); // NumberFormatException
            System.out.println("Parsed: " + value);
        } catch (NumberFormatException e) {
            System.out.println("NumberFormatException caught: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("---- Exception Hierarchy Demonstration ----");
        demoArithmetic();
        demoNullPointer();
        demoArrayIndex();
        demoNumberFormat();

        // Showing hierarchy: all of these are subclasses of RuntimeException,
        // which is a subclass of Exception, which is a subclass of Throwable.
        try {
            int[] arr = new int[2];
            arr[3] = 10 / 0;
        } catch (RuntimeException e) {
            // A single RuntimeException catch block can catch ArithmeticException,
            // ArrayIndexOutOfBoundsException, NumberFormatException, NullPointerException, etc.
            System.out.println("Caught via common parent RuntimeException: " +
```

```
                    e.getClass().getSimpleName() + " -> " + e.getMessage());
            }
        }
    }
```

Q2. Program with nested try-catch blocks generating different exceptions in the inner and outer blocks

```
public class NestedTryCatchDemo {

    public static void main(String[] args) {
        System.out.println("---- Nested Try-Catch Demonstration ----");

        try {
            System.out.println("Outer try block start");

            try {
                System.out.println("Inner try block start");
                int[] arr = new int[3];
                arr[4] = 100;              // Inner exception: ArrayIndexOutOfBoundsException
                System.out.println("Inner try block end");
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: ArrayIndexOutOfBoundsException - " + e.getMessage());
                // Deliberately generate another exception inside the inner catch
                int x = 10 / 0;            // ArithmeticException, will be caught by outer catch
            } finally {
                System.out.println("Inner finally block executed");
            }

            System.out.println("Outer try block end"); // not reached because inner catch threw
        } catch (ArithmeticException e) {
            System.out.println("Outer catch: ArithmeticException - " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Outer catch: Exception - " + e.getMessage());
        } finally {
            System.out.println("Outer finally block executed");
        }

        System.out.println("Program continues after nested try-catch");
    }
}
```

Q3. Calculator handling Division by zero, Invalid numeric input and Invalid operators with separate exception handlers

```
import java.util.Scanner;

public class CalculatorExceptionDemo {

    static double calculate(double num1, double num2, char operator) {
        switch (operator) {
            case '+': return num1 + num2;
            case '-': return num1 - num2;
            case '*': return num1 * num2;
            case '/':
                if (num2 == 0) {
                    throw new ArithmeticException("Division by zero is not allowed");
                }
                return num1 / num2;
            default:
                throw new IllegalArgumentException("Invalid operator: " + operator);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double num1 = 0, num2 = 0;
        char operator = ' ';
        boolean validInput = true;

        try {
            System.out.print("Enter first number: ");
            num1 = Double.parseDouble(sc.nextLine().trim());

            System.out.print("Enter second number: ");
            num2 = Double.parseDouble(sc.nextLine().trim());

            System.out.print("Enter operator (+, -, *, /): ");
            String opStr = sc.nextLine().trim();
            if (opStr.length() != 1) {
                throw new IllegalArgumentException("Operator must be a single character");
            }
            operator = opStr.charAt(0);

        } catch (NumberFormatException e) {
            System.out.println("Invalid numeric input: " + e.getMessage());
            validInput = false;
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid operator: " + e.getMessage());
            validInput = false;
        }

        if (validInput) {
            try {
                double result = calculate(num1, num2, operator);
                System.out.println("Result: " + result);
            } catch (ArithmeticException e) {
                System.out.println("Arithmetic error: " + e.getMessage());
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid operator: " + e.getMessage());
            }
        }

        sc.close();
    }
}
```

Q4. Student result-processing program with a custom InvalidMarksException

```
import java.util.Scanner;

// Custom exception
class InvalidMarksException extends Exception {
    public InvalidMarksException(String message) {
        super(message);
    }
}

public class StudentResultProcessing {

    static void validateMarks(int marks, String subject) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Invalid marks for " + subject +
                    ": " + marks + " (must be between 0 and 100)");
        }
    }

    static String getGrade(double percentage) {
        if (percentage >= 90) return "A+";
        else if (percentage >= 80) return "A";
        else if (percentage >= 70) return "B";
        else if (percentage >= 60) return "C";
        else if (percentage >= 50) return "D";
        else return "F";
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] subjects = {"Maths", "Physics", "Chemistry", "English", "Computer Science"};
        int[] marks = new int[subjects.length];
        boolean allValid = true;

        for (int i = 0; i < subjects.length; i++) {
            try {
                System.out.print("Enter marks for " + subjects[i] + ": ");
                marks[i] = Integer.parseInt(sc.nextLine().trim());
                validateMarks(marks[i], subjects[i]);
            } catch (NumberFormatException e) {
                System.out.println("Please enter a valid integer for " + subjects[i]);
                allValid = false;
            } catch (InvalidMarksException e) {
                System.out.println(e.getMessage());
                allValid = false;
            }
        }

        if (allValid) {
            int total = 0;
            for (int m : marks) total += m;
            double percentage = (double) total / subjects.length;
            String grade = getGrade(percentage);

            System.out.println("\n---- Result ----");
            System.out.println("Total: " + total);
            System.out.println("Percentage: " + percentage + "%");
            System.out.println("Grade: " + grade);
        } else {
            System.out.println("\nResult cannot be calculated due to invalid input. " +
                    "Please re-enter all marks correctly.");
        }

        sc.close();
    }
}
```

Q5. Banking application with InsufficientBalanceException, InvalidAmountException and AccountNotFoundException

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

// Custom exceptions
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) { super(message); }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) { super(message); }
}

class AccountNotFoundException extends Exception {
    public AccountNotFoundException(String message) { super(message); }
}

class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }

    public double getBalance() { return balance; }
    public String getAccountNumber() { return accountNumber; }

    // Lower-level method: validates amount and updates balance
    private void updateBalance(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Amount must be greater than zero");
        }
        balance += amount;
    }

    public void deposit(double amount) throws InvalidAmountException {
        updateBalance(amount);   // exception propagates upward if thrown
        System.out.println("Deposited: " + amount);
    }

    public void withdraw(double amount) throws InvalidAmountException, InsufficientBalanceException {
        if (amount <= 0) {
            throw new InvalidAmountException("Amount must be greater than zero");
        }
        if (amount > balance) {
            throw new InsufficientBalanceException(
                    "Insufficient balance. Current balance: " + balance + ", requested: " + amount);
        }
        updateBalance(-amount);
        System.out.println("Withdrawn: " + amount);
    }
}

public class BankingApplication {

    static Map<String, BankAccount> accounts = new HashMap<>();

    static BankAccount getAccount(String accNo) throws AccountNotFoundException {
        BankAccount acc = accounts.get(accNo);
        if (acc == null) {
            throw new AccountNotFoundException("No account found with number: " + accNo);
        }
        return acc;
    }

    public static void main(String[] args) {
```

```
        accounts.put("ACC001", new BankAccount("ACC001", 5000));
        accounts.put("ACC002", new BankAccount("ACC002", 1500));

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter account number: ");
        String accNo = sc.nextLine().trim();

        try {
            BankAccount acc = getAccount(accNo);

            System.out.print("Enter deposit amount: ");
            double depositAmt = Double.parseDouble(sc.nextLine().trim());
            acc.deposit(depositAmt);

            System.out.print("Enter withdrawal amount: ");
            double withdrawAmt = Double.parseDouble(sc.nextLine().trim());
            acc.withdraw(withdrawAmt);

            System.out.println("Final balance for " + acc.getAccountNumber() + ": " + acc.getBalance());

        } catch (AccountNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (InvalidAmountException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter a valid numeric amount");
        } finally {
            sc.close();
        }
    }
}
```

Q6. Login system with InvalidUsernameException, InvalidPasswordException and AccountLockedException

```
import java.util.Scanner;

// Custom exceptions
class InvalidUsernameException extends Exception {
    public InvalidUsernameException(String message) { super(message); }
}

class InvalidPasswordException extends Exception {
    public InvalidPasswordException(String message) { super(message); }
}

class AccountLockedException extends Exception {
    public AccountLockedException(String message) { super(message); }
}

public class LoginSystem {

    private static final String VALID_USERNAME = "admin";
    private static final String VALID_PASSWORD = "pass123";
    private static final int MAX_ATTEMPTS = 3;

    static void validateLogin(String username, String password)
            throws InvalidUsernameException, InvalidPasswordException {
        if (!username.equals(VALID_USERNAME)) {
            throw new InvalidUsernameException("Username does not exist: " + username);
        }
        if (!password.equals(VALID_PASSWORD)) {
            throw new InvalidPasswordException("Incorrect password for user: " + username);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int attempts = 0;
        boolean loggedIn = false;
        boolean locked = false;

        while (attempts < MAX_ATTEMPTS && !loggedIn) {
            try {
                System.out.print("Enter username: ");
                String username = sc.nextLine().trim();
                System.out.print("Enter password: ");
                String password = sc.nextLine().trim();

                validateLogin(username, password);

                System.out.println("Login successful. Welcome, " + username + "!");
                loggedIn = true;

            } catch (InvalidUsernameException e) {
                attempts++;
                System.out.println("Login failed: " + e.getMessage());
            } catch (InvalidPasswordException e) {
                attempts++;
                System.out.println("Login failed: " + e.getMessage());
            } finally {
                System.out.println("Attempt " + attempts + " of " + MAX_ATTEMPTS + " completed");
                if (!loggedIn && attempts >= MAX_ATTEMPTS) {
                    locked = true;
                }
            }
        }

        if (locked) {
            try {
                throw new AccountLockedException(
                        "Account locked after " + MAX_ATTEMPTS + " failed attempts. Try again later.");
            } catch (AccountLockedException e) {
```

```
                System.out.println(e.getMessage());
            }
        }

        sc.close();
    }
}
```

Q7. Shopping cart application using a complete custom exception hierarchy

```
import java.util.*;

// ---------- Exception Hierarchy ----------
class ApplicationException extends Exception {
    public ApplicationException(String message) { super(message); }
}

class ProductException extends ApplicationException {
    public ProductException(String message) { super(message); }
}
class ProductNotFoundException extends ProductException {
    public ProductNotFoundException(String message) { super(message); }
}
class OutOfStockException extends ProductException {
    public OutOfStockException(String message) { super(message); }
}

class PaymentException extends ApplicationException {
    public PaymentException(String message) { super(message); }
}
class InvalidPaymentException extends PaymentException {
    public InvalidPaymentException(String message) { super(message); }
}
class InsufficientFundsException extends PaymentException {
    public InsufficientFundsException(String message) { super(message); }
}

class OrderException extends ApplicationException {
    public OrderException(String message) { super(message); }
}
class EmptyCartException extends OrderException {
    public EmptyCartException(String message) { super(message); }
}

// ---------- Domain Classes ----------
class Product {
    String id, name;
    double price;
    int stock;

    Product(String id, String name, double price, int stock) {
        this.id = id; this.name = name; this.price = price; this.stock = stock;
    }
}

class CartItem {
    Product product;
    int quantity;
    CartItem(Product product, int quantity) { this.product = product; this.quantity = quantity; }
}

class ShoppingCart {
    Map<String, Product> catalog = new HashMap<>();
    List<CartItem> items = new ArrayList<>();
    double walletBalance = 2000;

    ShoppingCart() {
        catalog.put("P1", new Product("P1", "Laptop", 55000, 3));
        catalog.put("P2", new Product("P2", "Mouse", 500, 0));      // out of stock
        catalog.put("P3", new Product("P3", "Keyboard", 1200, 5));
    }

    Product searchProduct(String id) throws ProductNotFoundException {
        Product p = catalog.get(id);
        if (p == null) {
            throw new ProductNotFoundException("Product not found with id: " + id);
        }
        return p;
    }
```

```
    void addToCart(String id, int qty) throws ProductNotFoundException, OutOfStockException {
        Product p = searchProduct(id);
        if (p.stock < qty) {
            throw new OutOfStockException("Product '" + p.name + "' is out of stock. Available: " + p.stock);
        }
        p.stock -= qty;
        items.add(new CartItem(p, qty));
        System.out.println("Added to cart: " + p.name + " x" + qty);
    }

    void removeFromCart(String id) throws ProductNotFoundException {
        CartItem toRemove = null;
        for (CartItem item : items) {
            if (item.product.id.equals(id)) { toRemove = item; break; }
        }
        if (toRemove == null) {
            throw new ProductNotFoundException("Cannot remove. Product not in cart: " + id);
        }
        items.remove(toRemove);
        System.out.println("Removed from cart: " + toRemove.product.name);
    }

    double calculateTotal() throws EmptyCartException {
        if (items.isEmpty()) {
            throw new EmptyCartException("Cart is empty. Add items before checkout.");
        }
        double total = 0;
        for (CartItem item : items) {
            total += item.product.price * item.quantity;
        }
        return total;
    }

    void makePayment(double amount, String paymentMode)
            throws InvalidPaymentException, InsufficientFundsException {
        if (!paymentMode.equalsIgnoreCase("wallet") && !paymentMode.equalsIgnoreCase("card")) {
            throw new InvalidPaymentException("Invalid payment mode: " + paymentMode);
        }
        if (paymentMode.equalsIgnoreCase("wallet") && amount > walletBalance) {
            throw new InsufficientFundsException(
                    "Insufficient wallet balance. Balance: " + walletBalance + ", required: " + amount);
        }
        if (paymentMode.equalsIgnoreCase("wallet")) {
            walletBalance -= amount;
        }
        System.out.println("Payment of " + amount + " successful via " + paymentMode);
    }

    void checkout(String paymentMode) {
        try {
            double total = calculateTotal();
            System.out.println("Order total: " + total);
            makePayment(total, paymentMode);
            System.out.println("Order placed successfully!");
        } catch (EmptyCartException e) {
            System.out.println("Order failed: " + e.getMessage());
        } catch (InvalidPaymentException | InsufficientFundsException e) {
            System.out.println("Payment failed: " + e.getMessage());
        }
    }
}

public class ShoppingCartApp {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        // Search for a product
        try {
            Product p = cart.searchProduct("P1");
            System.out.println("Found product: " + p.name + " @ " + p.price);
        } catch (ProductNotFoundException e) {
```

```
            System.out.println(e.getMessage());
        }

        // Add valid item
        try {
            cart.addToCart("P1", 1);
        } catch (ProductException e) {
            // catches both ProductNotFoundException and OutOfStockException
            System.out.println("Add to cart failed: " + e.getMessage());
        }

        // Try adding out-of-stock item
        try {
            cart.addToCart("P2", 1);
        } catch (ProductException e) {
            System.out.println("Add to cart failed: " + e.getMessage());
        }

        // Try adding non-existent product
        try {
            cart.addToCart("P9", 1);
        } catch (ProductException e) {
            System.out.println("Add to cart failed: " + e.getMessage());
        }

        // Add another valid item
        try {
            cart.addToCart("P3", 2);
        } catch (ProductException e) {
            System.out.println("Add to cart failed: " + e.getMessage());
        }

        // Remove an item that isn't in the cart
        try {
            cart.removeFromCart("P2");
        } catch (ProductNotFoundException e) {
            System.out.println(e.getMessage());
        }

        // Checkout - this will fail (wallet balance is lower than total)
        cart.checkout("wallet");

        // Try an empty cart checkout
        ShoppingCart emptyCart = new ShoppingCart();
        emptyCart.checkout("card");

        // Try an invalid payment mode
        try {
            cart.addToCart("P3", 1);
        } catch (ProductException e) {
            System.out.println("Add to cart failed: " + e.getMessage());
        }
        cart.checkout("upi"); // invalid mode
    }
}
```